Technical Report
VGA

Name: Al-Moataz Billah Alaa El-Dien Mohammed Ramadan

Introduction

Video Graphics Array (VGA) is an analog computer display standard introduced in 1987 by IBM, utilizing a 15-pin connector to transmit video signals. Although largely replaced by digital standards like HDMI and DisplayPort, it is still used for legacy hardware, supporting common resolutions up to 640 x 480 pixels.

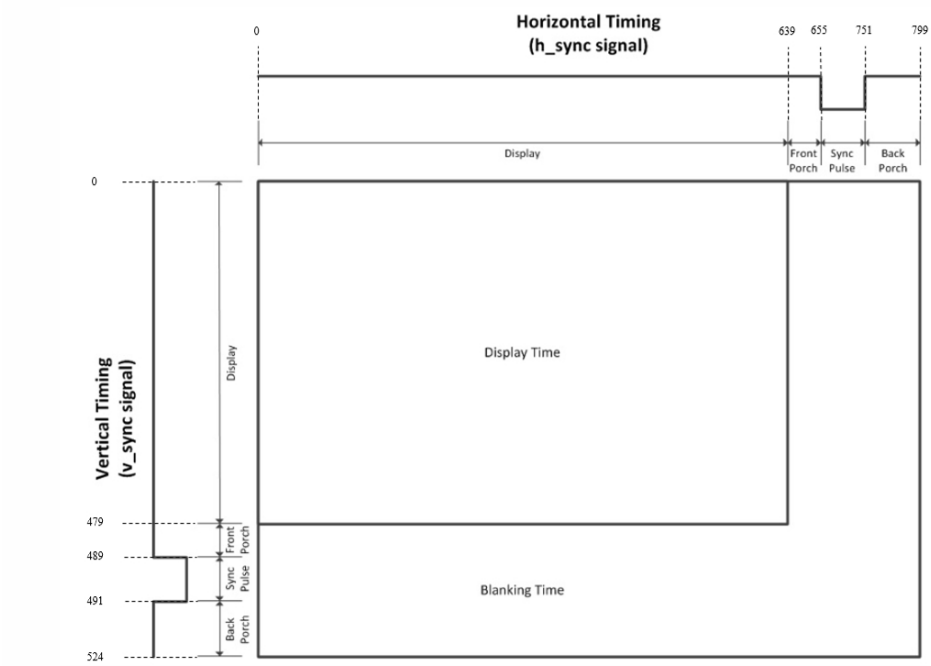
Discussion

Our project is designing the VGA controller with resolution 640 x 480 60Hz, so we have 640 pixel per line and 480 line per screen, and we want to display a video with 60 frame per second so the clock frequency will be

$$F_{clk} = 800\frac{pixel}{line}\ x\ 525\frac{line}{frame}\ x\ 60\frac{frame}{second} = 25.2\ MHz$$

$$\therefore T = 40\ ns$$

The CRT starts to shoot electrons to the screen line by line, it starts by the first line and then the second and so on, the screen has areas where the CRT doesn't shoot electrons to allow for CRT to retrace and start in the next line



This diagram shows the timing diagram of the signals and where the display happens and where the blanking timing occurs.

Verilog code for design

```
module VGA_controller #(parameter h_active      = 640,
                                  h_front_porch = 16,
                                  h_sync_porch  = 96,
                                  h_back_porch  = 48,
                                  v_active      = 480,
                                  v_front_porch = 10,
                                  v_sync_porch  = 2,
                                  v_back_porch  = 33,
                                  BPP           = 8
            )
(
    input clk, rstn,
    output reg [BPP-1:0] rgb,
    output h_sync, v_sync, video_on
);
localparam h_sync_width = h_active + h_front_porch + h_back_porch +
h_sync_porch;
localparam v_sync_width = v_active + v_front_porch + v_back_porch +
v_sync_porch;
localparam ADDR_WIDTH   = $clog2(v_active * h_active);
localparam pixel_x_bits = $clog2(h_sync_width);
localparam pixel_y_bits = $clog2(v_sync_width);

wire [pixel_x_bits-1:0] pixel_x;
wire [pixel_y_bits-1:0] pixel_y;
wire tick_x, tick_y;
wire [ADDR_WIDTH - 1:0] addr;


modulus_counter #(.M(h_sync_width))
pixels_x (
.reset(rstn),
.clk(clk),
.enable(1'b1),
.count(pixel_x),
.done(tick_x)
);
modulus_counter #(.M(v_sync_width))
pixels_y (
.reset(rstn),
```

```
.clk(clk),
.enable(tick_x),
.count(pixel_y),
.done(tick_y)
);



// assuming that the photos that will be displayed will be exactly the same
// in wdith and height as the screen which is 640 x 480 and each pixel has
// a color and each color is represented with BPP bits so the word will have
// 8 bits for color, and each line will represent one pixel, and each pixel is
// represented by one line, so number lines equal to total number of pixels
// = 640 * 480 = 307200, this number will change if the pixels changes

reg [BPP-1:0] memory [0 : 2 ** ADDR_WIDTH - 1];
assign video_on = (pixel_x < h_active  && pixel_y < v_active)? 1'b1 : 1'b0;
assign h_sync = (pixel_x <(h_active + h_front_porch - 1) || pixel_x
>=(h_active + h_front_porch + h_sync_porch - 1))? 1'b1 : 1'b0;
assign v_sync = (pixel_y <(v_active + v_front_porch - 1) || pixel_y
>=(v_active + v_front_porch + v_sync_porch - 1))? 1'b1 : 1'b0;

// addr = number of columns * counter_of_rows + counter_of_columns
assign addr = v_active * pixel_y  + pixel_x;

always @(posedge clk)
begin
    if(video_on)
        rgb = memory[addr];
    else
        rgb = rgb;
end



endmodule
```

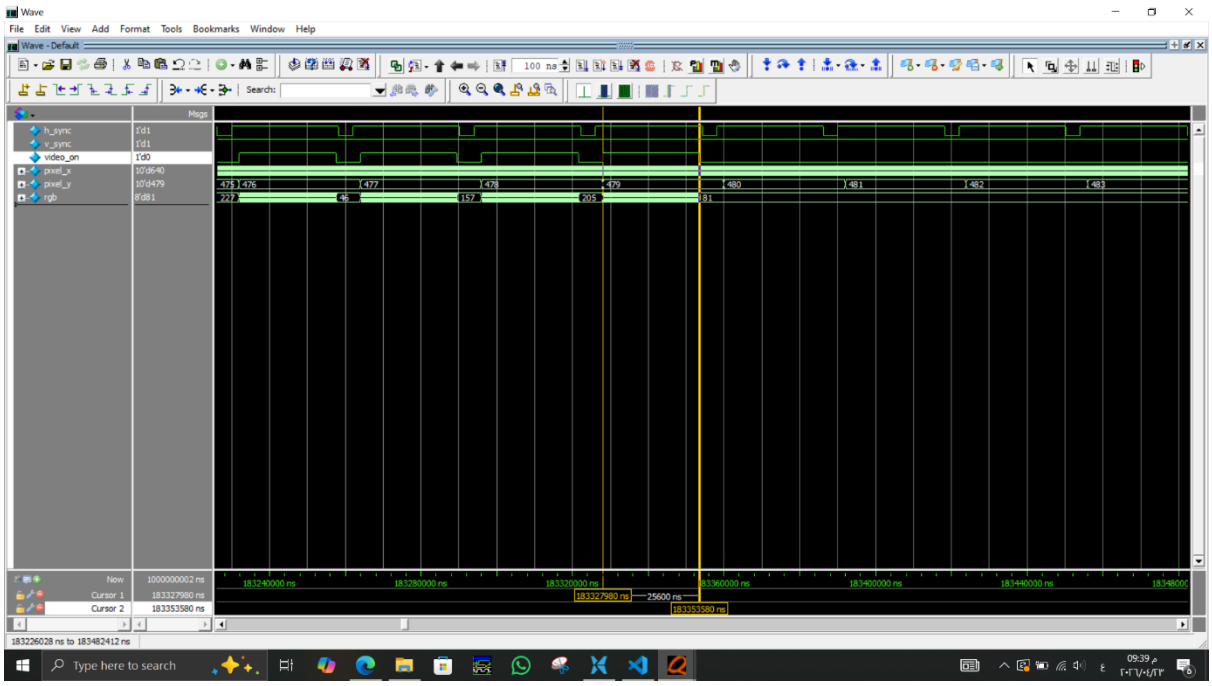
```
module modulus_counter #(parameter M = 800) (
    input reset, clk, enable,
    output [$clog2(M)-1:0] count,
    output done
);
reg [$clog2(M)-1:0] q_reg, q_next;
always @(posedge clk, negedge reset)
begin
    if(!reset)
        q_reg <= 'b0;
    else if(enable)
```

```
        q_reg <= q_next;
    else
        q_reg <= q_reg;
end
assign done = (q_reg == M-1);
always @(*)
begin
    if(done)
        q_next = 'b0;
    else
        q_next = q_reg + 1;
end

assign count = q_reg;

endmodule
```

```
module VGA_tb;
// Parameters
localparam  h_active      = 640,
            h_front_porch = 16,
            h_sync_porch  = 96,
            h_back_porch  = 48,
            v_active      = 480,
            v_front_porch = 10,
            v_sync_porch  = 2,
            v_back_porch  = 33,
            BPP           = 8,
            T             = 40;
// Ports
reg clk, reset;
wire [BPP - 1:0] rgb;
wire h_sync, v_sync, video_on;


VGA_controller # (
.h_active(h_active),
.h_front_porch(h_front_porch),
.h_sync_porch(h_sync_porch),
.h_back_porch(h_back_porch),
.v_active(v_active),
.v_front_porch(v_front_porch),
.v_sync_porch(v_sync_porch),
.v_back_porch(v_back_porch),
.BPP(BPP)
)
VGA_inst (
.clk(clk),
```

```
.rstn(reset),
.rgb(rgb),
.h_sync(h_sync),
.v_sync(v_sync),
.video_on(video_on)
);

always  begin
clk = 0;
#(T/2);

clk = 1;
#(T/2);
end

initial begin
    reset = 0;
    #2;
    reset = 1;
    $readmemb("data.mem", VGA_inst.memory);

    #(10**9);
    $stop;


end


endmodule
```

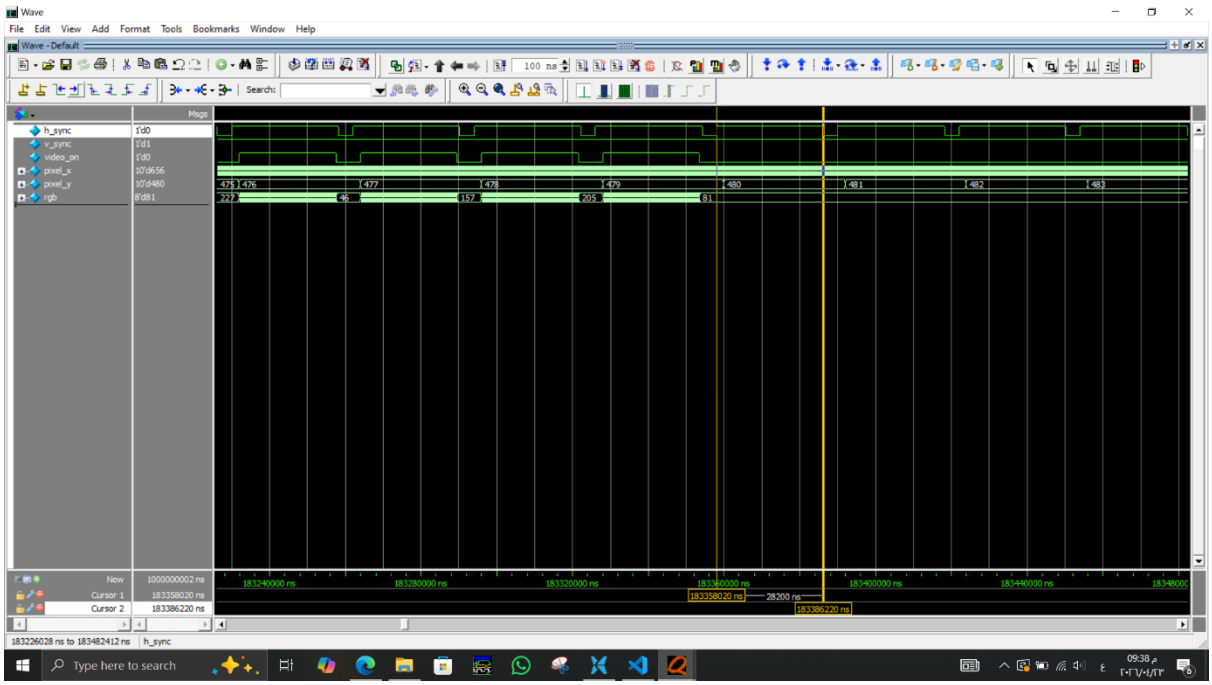
Waveforms



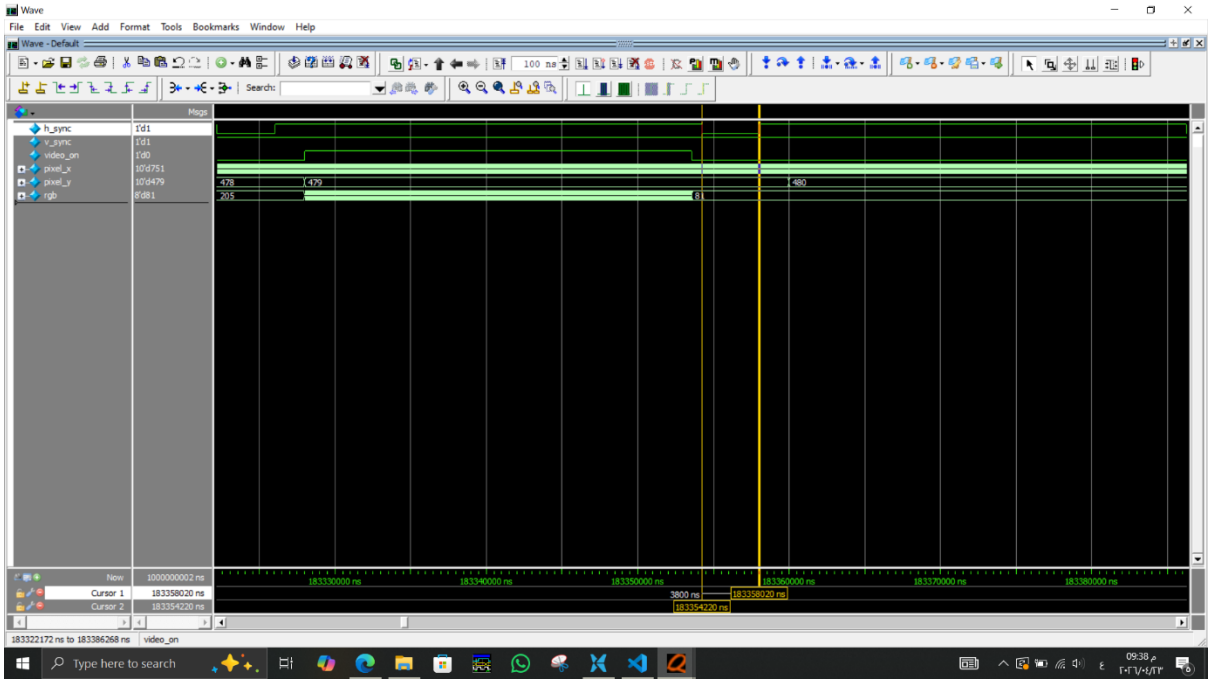
This shows the video_on signal which is high when we are in the display area, to make sure, here is the calculation

$$display\ time = \frac{width\ of\ the\ high\ period\ of\ video\ on\ signal}{period\ of\ clock} = \frac{25600\ n}{40\ n} = 640$$

Which is the same as the value of pixels per line as displayed in timing graph.



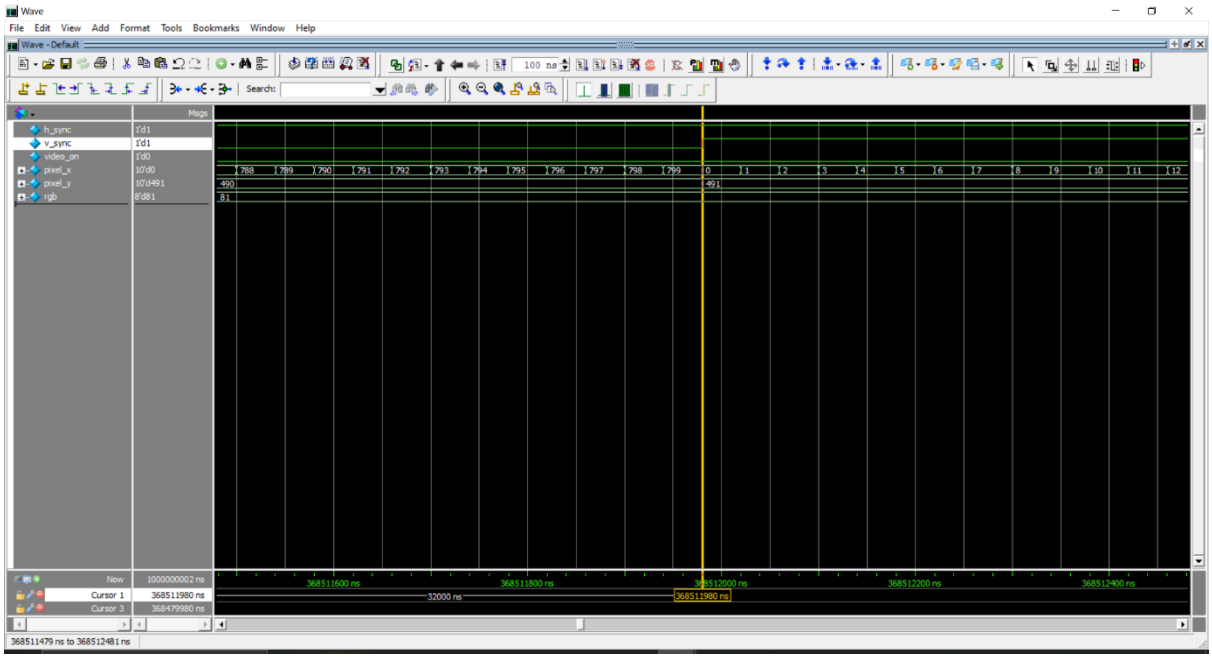
This photo shows that the h_sync signal is high for the intended period

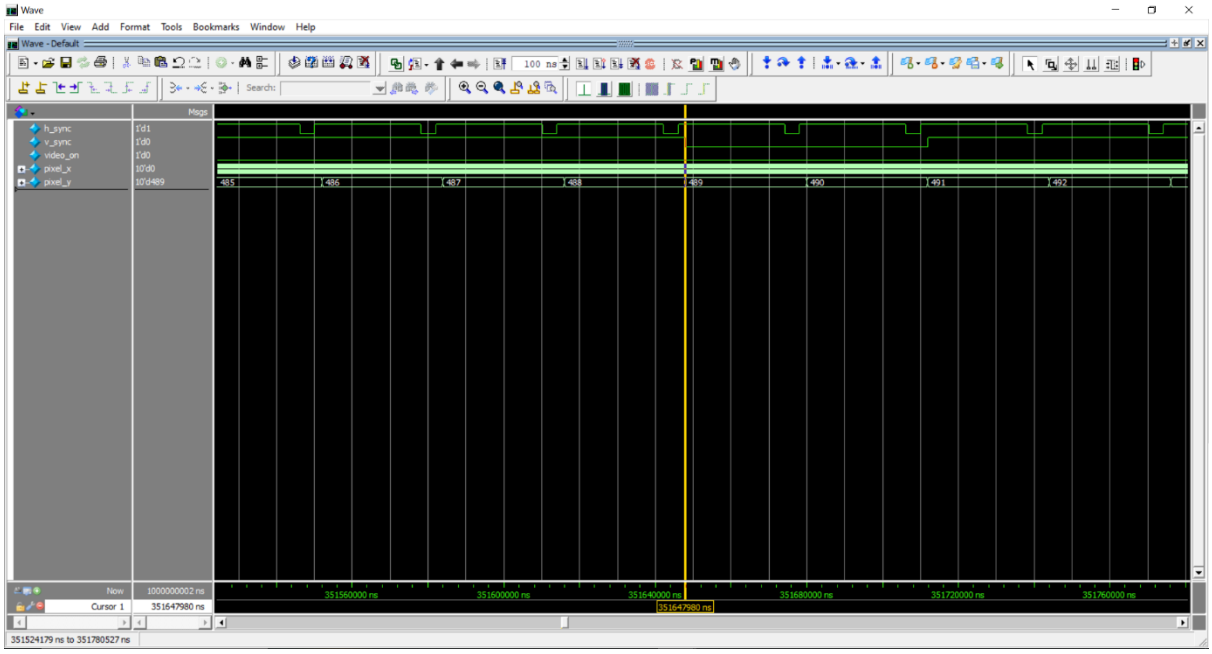


$$width\ of\ hsync\ signal = \frac{28200}{40} = 705\ s$$

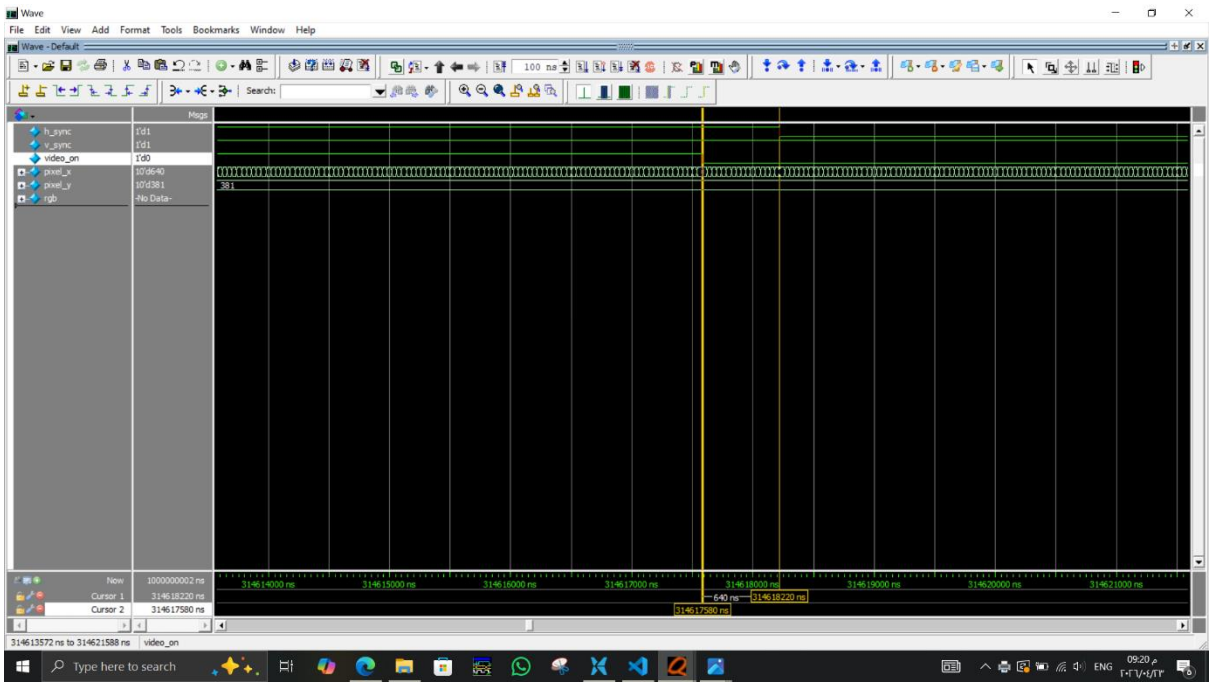
$$width\ of\ the\ off\ period = \frac{3800}{40} = 95\ s$$

The signal lasts for the intended period as specified by the standards





These two images shows that v_sync signals goes high and low when pixel_y reach 491 and 489 respectively, which assures again that our design works as the standard specifies.



The video_on signal becomes low before h_sync becomes low by the amount of h_front_porch $h_front_porch = \frac{640}{40} = 16$